Отчет
о выполнении лабораторной работы № 4
Глубокие сети прямого распространения
Вариант 13

Выполнил студент группы ЗФИбд-01-24: Князев А. М.

Проверил: Леонов С. С.

Москва, 2026

Цель работы

изучение глубоких полносвязных нейронных сетей в задачах бинарной классификации, многоклассовой классификации и регрессии.

Основные этапы работы

1. Построить бинарный классификатор отзывов IMDB.
2. Построить многоклассовый классификатор.
3. Построить регрессионную модель с проверкой по K блокам.

Ход выполнения и результаты

Таблица 1 — Архитектуры варианта

Этап | Архитектура | Функция потерь

IMDB | 16-32-16, relu-relu-tanh | MSE

Базовый IMDB | 16-16, relu-relu | BCEWithLogitsLoss

Многокласс | 32-64-64, tanh-mish-relu | CrossEntropyLoss

Регрессия | 32-64-64, tanh-mish-relu | MSE

Таблица 2 — Результаты проверки

Задача | Метрика

IMDB variant | test_loss = 0.0980, test_acc = 0.6179

IMDB baseline | test_loss = 0.2494, test_acc = 0.8946

Multiclass | test_loss = 0.0442, test_acc = 1.0000

Regression | MAE = [55.36, 46.31, 41.19, 59.58], mean = 50.61

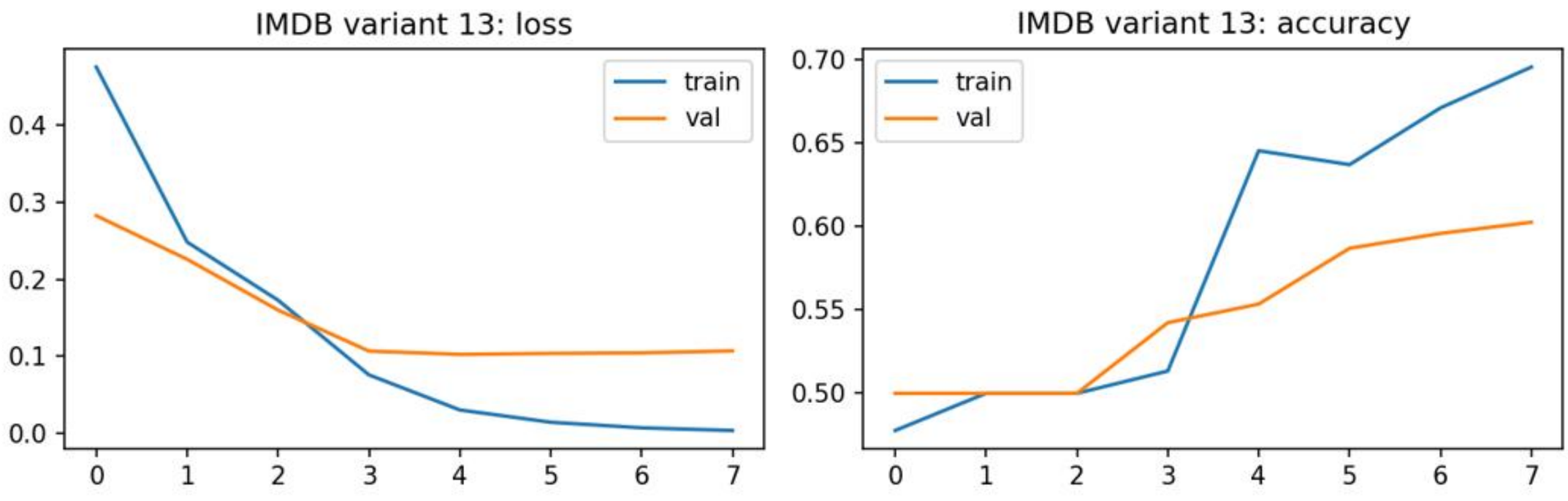


Рис. 1 — Графики loss и accuracy для IMDB

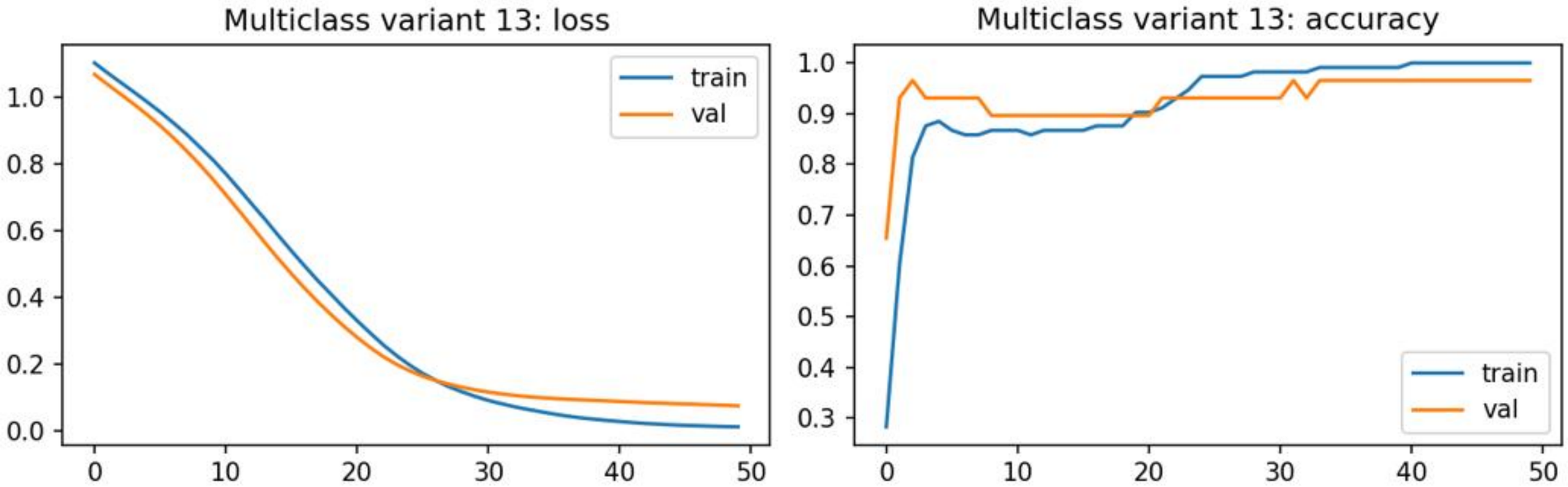


Рис. 2 — Графики loss и accuracy для многоклассовой задачи

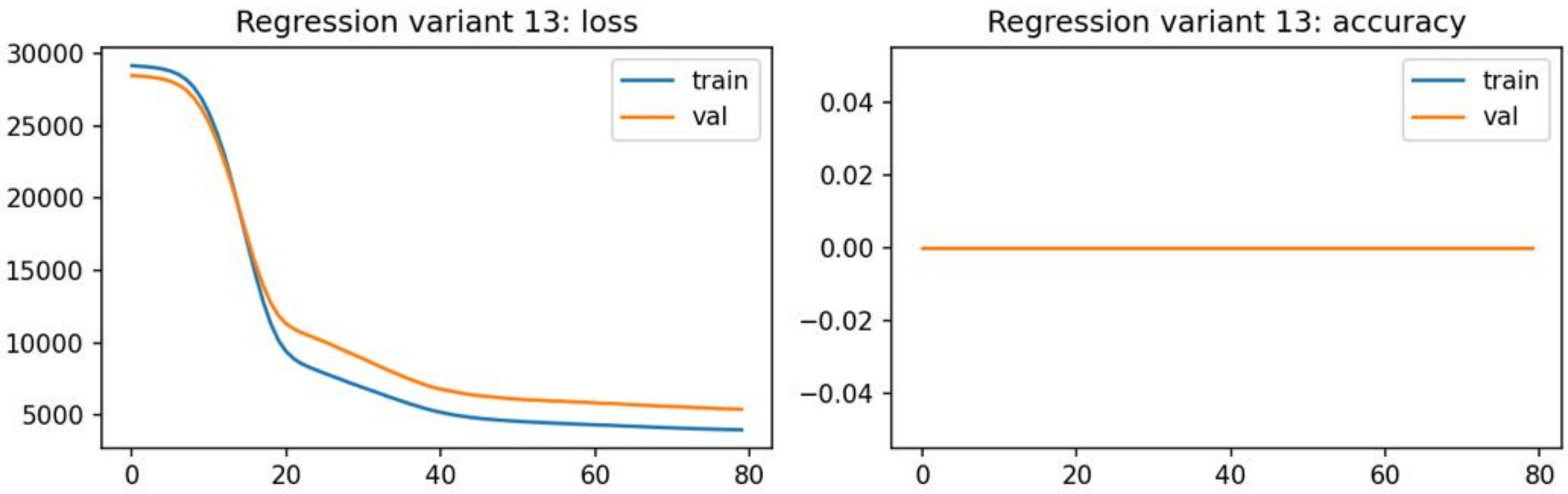


Рис. 3 — Графики обучения регрессионной модели

Вывод

Полносвязные сети успешно применены к трем типам задач. Базовый IM
классификатор оказался точнее варианта с MSE, что ожидаемо для бин
классификации, где логистическая кросс-энтропия лучше согласована
задачей.

Фрагменты кода из main.ipynb

```
from __future__ import annotations


import math

import random

import zipfile

from collections import Counter

from pathlib import Path
```

```
import matplotlib.pyplot as plt

import numpy as np

import torch

import torch.nn as nn

from sklearn.datasets import load_breast_cancer, load_diabetes, load_wine

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.model_selection import KFold, train_test_split

from sklearn.preprocessing import StandardScaler

from torch.utils.data import DataLoader, TensorDataset
```

```


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

SEED = 13

random.seed(SEED)

np.random.seed(SEED)

torch.manual_seed(SEED)
```

```


def activation(name: str) -> nn.Module:

    return {

        "relu": nn.ReLU(),

        "tanh": nn.Tanh(),

        "mish": nn.Mish(),

        "elu": nn.ELU(),

        "leaky_relu": nn.LeakyReLU(0.1),

    }[name]
```

```


def make_mlp(input_size: int, hidden: list[int], activations: list[str], output_size: int) -> nn.Sequentia

    layers: list[nn.Module] = []

    prev = input_size

    for width, act in zip(hidden, activations):

        layers.append(nn.Linear(prev, width))

        layers.append(activation(act))
```

```
        prev = width
    layers.append(nn.Linear(prev, output_size))
    return nn.Sequential(*layers)


def train_model(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    criterion: nn.Module,
    epochs: int = 8,
    lr: float = 1e-3,
    classification: str = "binary",
) -> dict[str, list[float]]:
    model.to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}
    for _ in range(epochs):
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0
        for x, y in train_loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()
            train_loss += float(loss.item()) * len(x)
            train_correct += count_correct(logits, y, classification)
            train_total += len(x)
        val_loss, val_acc = evaluate(model, val_loader, criterion, classification)
        history["loss"].append(train_loss / train_total)
        history["acc"].append(train_correct / train_total)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
    return history


def count_correct(logits: torch.Tensor, y: torch.Tensor, classification: str) -> int:
    if classification == "multiclass":
        return int((logits.argmax(dim=1) == y).sum().item())
    pred = (torch.sigmoid(logits).view(-1) >= 0.5).float()
```

```
    return int((pred == y.view(-1)).sum().item())


def evaluate(model: nn.Module, loader: DataLoader, criterion: nn.Module, classification: str) -> tuple[flo
float]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            loss = criterion(logits, y)
            total_loss += float(loss.item()) * len(x)
            correct += count_correct(logits, y, classification)
            total += len(x)
    return total_loss / total, correct / total


def make_loader(x: np.ndarray, y: np.ndarray, batch_size: int = 64) -> DataLoader:
    dataset = TensorDataset(torch.tensor(x, dtype=torch.float32), torch.tensor(y))
    return DataLoader(dataset, batch_size=batch_size, shuffle=True)


def read_imdb(zip_path: Path, max_docs_per_class: int = 700) -> tuple[list[str], np.ndarray]:
    texts: list[str] = []
    labels: list[int] = []
    with zipfile.ZipFile(zip_path) as archive:
        for split in ["train", "test"]:
            for label_name, label in [("neg", 0), ("pos", 1)]:
                prefix = f"aclImdb/{split}/{label_name}/"
                names = [n for n in archive.namelist() if n.startswith(prefix) and n.endswith(".txt")]
                for name in names[:max_docs_per_class]:
                    texts.append(archive.read(name).decode("utf-8", errors="ignore"))
                    labels.append(label)
    return texts, np.array(labels, dtype=np.float32)


def plot_history(history: dict[str, list[float]], path: Path, title: str) -> None:
    fig, axes = plt.subplots(1, 2, figsize=(9, 3))
    axes[0].plot(history["loss"], label="train")
    axes[0].plot(history["val_loss"], label="val")
    axes[0].set_title(f"{title}: loss")
    axes[0].legend()
    axes[1].plot(history["acc"], label="train")
```

```
    axes[1].plot(history["val_acc"], label="val")
    axes[1].set_title(f"{title}: accuracy")
    axes[1].legend()
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)


def stage_imdb() -> None:
    texts, y = read_imdb(Path("IMDB/IMDB/aclImdb.zip"))
    x_train_text, x_test_text, y_train, y_test = train_test_split(texts, y, test_size=0.2, stratify=y,
random_state=SEED
    x_train_text, x_val_text, y_train, y_val = train_test_split(
        x_train_text, y_train, test_size=0.2, stratify=y_train, random_state=SEED
    )
    vectorizer = CountVectorizer(max_features=5000, binary=True, stop_words="english")
    x_train = vectorizer.fit_transform(x_train_text).astype("float32").toarray()
    x_val = vectorizer.transform(x_val_text).astype("float32").toarray()
    x_test = vectorizer.transform(x_test_text).astype("float32").toarray()

    model = make_mlp(x_train.shape[1], [16, 32, 16], ["relu", "relu", "tanh"], 1)
    baseline = make_mlp(x_train.shape[1], [16, 16], ["relu", "relu"], 1)
    criterion = nn.MSELoss()

    def wrap_targets(values: np.ndarray) -> np.ndarray:
        return values.reshape(-1, 1).astype("float32")

    train_loader = make_loader(x_train, wrap_targets(y_train))
    val_loader = make_loader(x_val, wrap_targets(y_val))
    test_loader = make_loader(x_test, wrap_targets(y_test), batch_size=128)

    history = train_model(model, train_loader, val_loader, criterion, epochs=8, classification="binary")
    base_history = train_model(baseline, train_loader, val_loader, nn.BCEWithLogitsLoss(), epochs=8,
classification="bin
    plot_history(history, Path("lab_4/imdb_history.png"), "IMDB variant 13")
    loss, acc = evaluate(model, test_loader, criterion, "binary")
    base_loss, base_acc = evaluate(baseline, test_loader, nn.BCEWithLogitsLoss(), "binary")
    print(f"IMDB variant: test_loss={loss:.4f}, test_acc={acc:.4f}")
    print(f"IMDB baseline: test_loss={base_loss:.4f}, test_acc={base_acc:.4f}")
```